Session plan

Section	Topic
0	Recap: what Spring MVC made us do
1	What Spring Boot is (and isn't) — the 3 pillars, at a glance
2	Pillar 1 — Starters + the BOM
3	Pillar 2 — Auto-configuration (+ convention over configuration)
4	Pillar 3 — Embedded server & executable jar
5	Worked examples
6	The main class and what happens at startup
7	`application.yml` — the basics

---

0. Recap — what Spring MVC made us do

In the Spring MVC session, to get *one* JSON endpoint working we had to touch a lot of things that had **nothing to do with our business logic**:

- Register the `DispatcherServlet` (in `web.xml` or a `WebApplicationInitializer`).
- Turn on MVC with `@EnableWebMvc` and configure a `WebMvcConfigurer`.
- Set up the **context hierarchy**: an optional root context plus a web context for each `DispatcherServlet`, and decide which beans live in which.
- Add the Jackson dependency yourself, at a version that matches your Spring version (MVC then detects it and registers the JSON converter for you).
- Choose compatible versions of Spring, Jackson, Hibernate, Servlet API, logging…
- Install a Tomcat, build a **WAR**, and deploy it.

The plumbing is **repetitive** and **mostly identical across projects**. Spring Boot's whole purpose is to take the plumbing off your hands **without removing your ability to change it**.

---

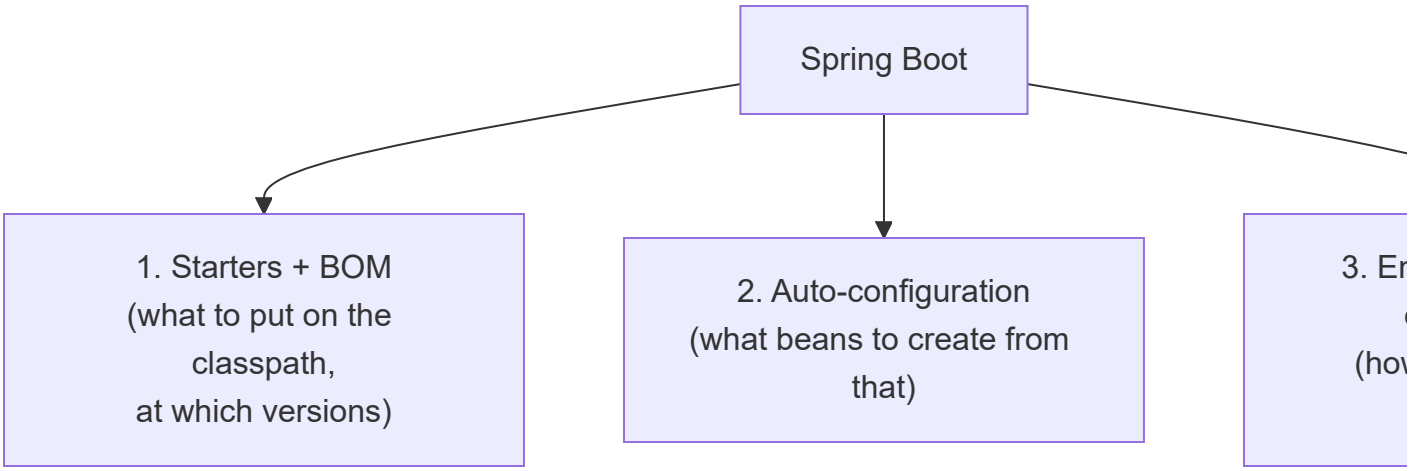
1. What Spring Boot is (and isn't)

> **is a project that makes it much easier to build standalone, production-ready applications with the Spring framework. It's built on top of the core Spring Framework but removes a lot of the setup pain that used to come with it.**

It is **not**:

Misconception	Reality
"A new framework that replaces Spring"	It **is** Spring underneath. Same `ApplicationContext`, same beans, same MVC.
"Replaces Spring MVC"	Boot *configures* Spring MVC for you. Your `@RestController` code is identical.
"Code generation"	Nothing is generated. Boot creates **beans at startup** based on the classpath and properties.
"Only for microservices"	It is fine for monoliths too. It is a way of *building and running* apps.
"Magic you cannot control"	Every default **backs off** when you define your own bean or property. You'll see how.

The three pillars — the map for this whole session



Spring MVC vs Spring Boot — the same job, side by side

Concern	Plain Spring MVC (typical)	Spring Boot
Servlet registration	`web.xml` / `WebApplicationInitializer`	Auto-configured `DispatcherServlet`
Application contexts	Optional root context + one child context per `DispatcherServlet`	**One** context (see section 6.3)

Concern	Plain Spring MVC (typical)	Spring Boot
Enable MVC	`@EnableWebMvc` + config class	Auto-configured; add a `WebMvcConfigurer` only to customize
JSON conversion	Add the Jackson dependency yourself; MVC detects it and registers the converter	Jackson comes with the starter; a `JsonMapper` **bean** is preconfigured, tunable with `spring.jackson.*`
Dependency versions	Chosen and aligned by hand	One **BOM** (bill of materials) manages them
Server	Install Tomcat, deploy WAR	Embedded Tomcat, run `java -jar app.jar`
Config values	`@PropertySource`, placeholder configurer	`application.yml`
Logging	Set it up yourself	Logback preconfigured (Session 2)
Health/metrics	Build yourself	Actuator (Session 2)

> ⑦ **Can I use Spring Boot to serve JSP pages, like classic Spring MVC? Technically yes, but it breaks pillar 3: JSPs need to be compiled from files on a real filesystem, and that doesn't work reliably from inside an executable jar. To serve JSP from Boot you'd have to go back to WAR packaging + an external Tomcat — which throws away "just run the jar." You'd still use Spring Boot for everything else (pillars 1 and 2 still apply), but for server-rendered pages the idiomatic Boot choice is a template engine that works from the classpath, like Thymeleaf — not JSP. Nobody chooses plain Spring MVC over Boot just to keep JSP; they just don't use JSP anymore.**

---

2. Pillar 1 — Starters + dependency management

2.1 What a starter is

> 🔥 **A starter is just a dependency descriptor It contains no code (or almost none). It is a `pom.xml` that says *"if you want to do X, you need these libraries, at these compatible versions."***

```
spring-boot-starter-webmvc          (simplified — run dependency:tree for the
real one)
```

```
  ├─ spring-boot-starter                (core: logging, YAML, auto-config base)
  ├─ spring-boot-starter-jackson        (JSON)
  ├─ spring-boot-starter-tomcat         (embedded server)
  └─ spring-webmvc                      (Spring MVC itself)
```

See it yourself:

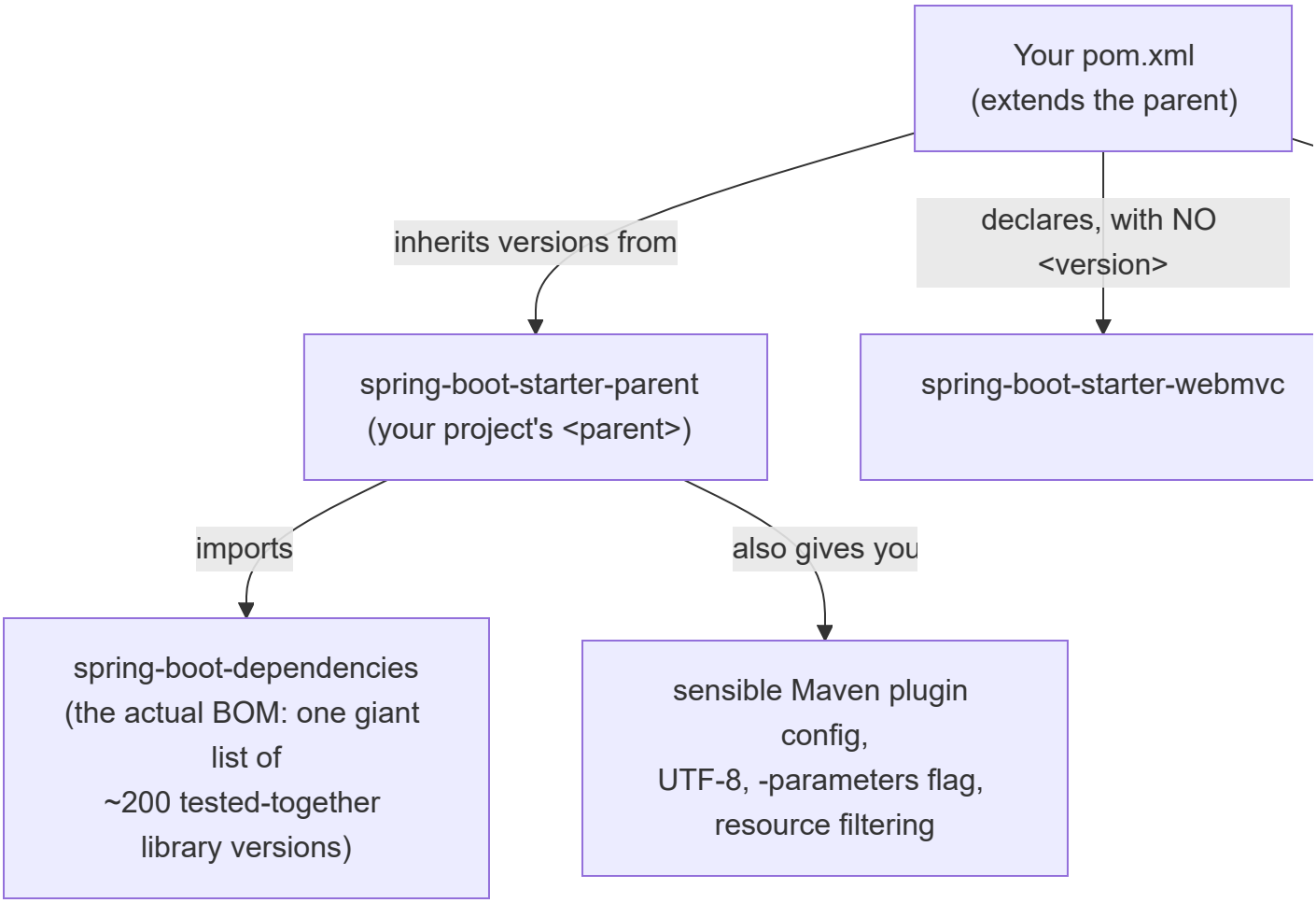
```
./mvnw dependency:tree
```

2.2 Common starters (Boot 4 names)

Need	Starter	Test companion
REST / web (servlet stack)	`spring-boot-starter-webmvc`	`…-webmvc-test`
Reactive web	`spring-boot-starter-webflux`	`…-webflux-test`
Database with JPA/Hibernate	`spring-boot-starter-data-jpa`	`…-data-jpa-test`
Plain JDBC	`spring-boot-starter-jdbc`	`…-jdbc-test`
Bean validation	`spring-boot-starter-validation`	`…-validation-test`
Security	`spring-boot-starter-security`	`…-security-test`
Health/metrics endpoints	`spring-boot-starter-actuator`	`…-actuator-test`
Outgoing HTTP (`RestClient`)	`spring-boot-starter-restclient`	`…-restclient-test`
Schema migrations	`spring-boot-starter-flyway` / `…-liquibase`	—
Templates (server-rendered HTML — see the JSP callout above)	`spring-boot-starter-thymeleaf`	`…-thymeleaf-test`
Kafka / AMQP	`spring-boot-starter-kafka` / `…-amqp`	`…-kafka-test` / `…-amqp-test`

2.3 Where the versions actually come from — parent, dependencies, BOM

This is the piece that's easy to gloss over, so let's be explicit about who does what.



So there are really two separate jobs, that happen to travel together:

1. **Starters** decide *which* libraries you need (pillar 1, section 2.1–2.2).
2. **The BOM** decides *which version* of each one to use — and it's inherited, not something you write.

Your `pom.xml`'s `<parent>` is `spring-boot-starter-parent`, and that parent itself has a `<parent>` — `spring-boot-dependencies`. *That's* the BOM: a `<dependencyManagement>` block pinning a compatible version for every library Boot knows about. That's why this compiles with no `<version>` at all:

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>            <!-- no <version> -->
    <scope>runtime</scope>
</dependency>
```

Override a version only deliberately, via a property (e.g. `<h2.version>…</h2.version>`), and only when you have a reason (a security fix, a needed feature).

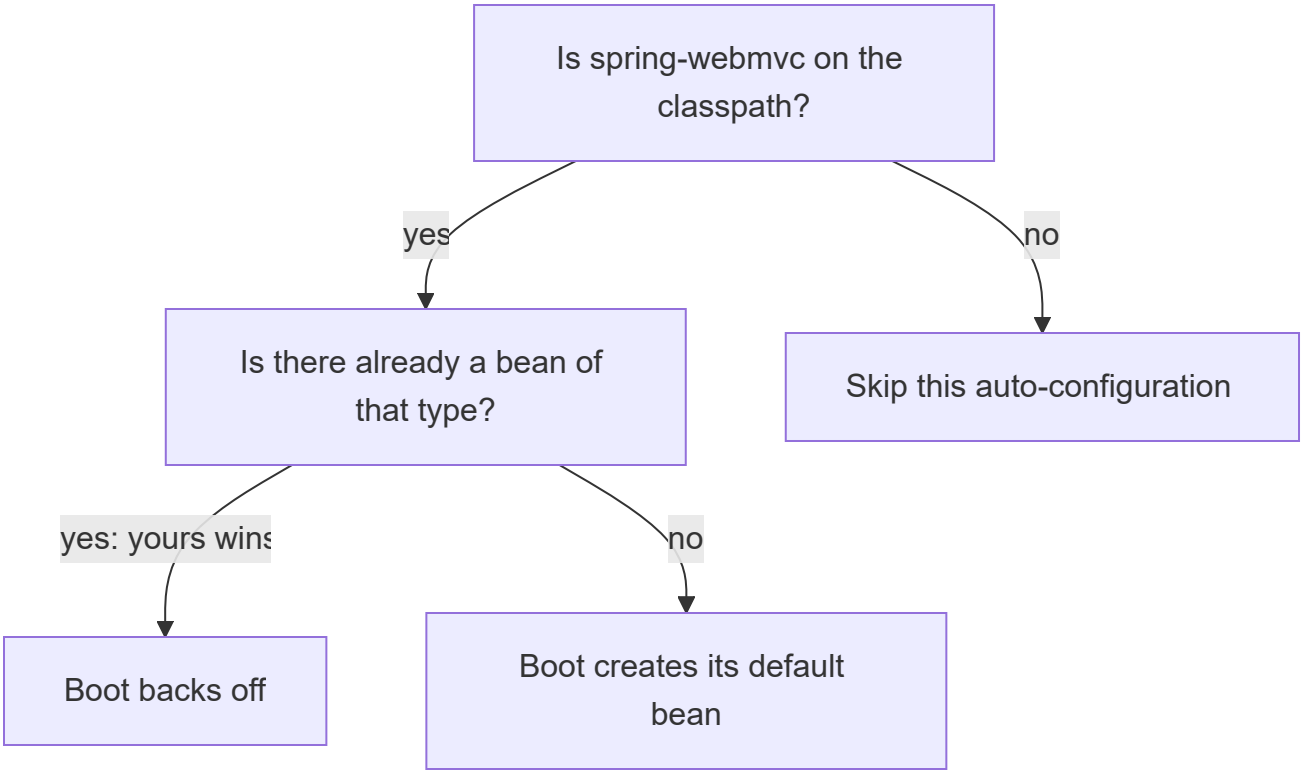
---

3. Pillar 2 — Auto-configuration

> Definition **Auto-configuration** = *Boot looks at what is on your classpath (pillar 1) and in your properties, and creates the beans you would probably have written yourself.*

3.1 The rule that makes it safe: **back off**

Auto-configured beans are **defaults**. The moment you define your own, Boot's version **steps aside**.



This is implemented with **conditions**. A simplified illustration (not the real source):

```
@AutoConfiguration
@ConditionalOnClass(JsonMapper.class)                // only if Jackson is on
the classpath
public class JacksonAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                        // only if the developer
didn't define one
    JsonMapper jsonMapper(JsonMapper.Builder builder) {
        return builder.build();
    }
}
```

Condition	Meaning
`@ConditionalOnClass`	"A given class is on the classpath" → the library is present
`@ConditionalOnMissingClass`	…it is **not** present
`@ConditionalOnMissingBean`	"The user has not defined this bean" → **back off**
`@ConditionalOnBean`	"Another bean exists"
`@ConditionalOnProperty`	"A property has a given value"
`@ConditionalOnWebApplication`	"It's a servlet/reactive web app"

3.2 Where does Boot find the auto-configurations?

Every Boot module ships a text file listing fully-qualified class names:

```
META-
INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

```
org.springframework.boot.autoconfigure.jackson.JacksonAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
org.springframework.boot.autoconfigure.kafka.KafkaAutoConfiguration
...
```

How it's processed:

1. `@EnableAutoConfiguration` triggers Spring to collect these `.imports` files from every JAR on the classpath
2. Each listed class is loaded and its conditions are checked (`@ConditionalOnClass` , `@ConditionalOnMissingBean` , `@ConditionalOnProperty` , etc.)
3. **Conditions pass** → the class's `@Bean` methods activate, beans get registered
4. **Conditions fail** → the class is skipped entirely, no beans created

In practice: most candidates fail and get skipped — you don't have Kafka on the classpath, so `KafkaAutoConfiguration` 's `@ConditionalOnClass` check fails, no Kafka beans. Only the handful matching your actual dependencies pass and contribute beans.

3.3 Convention over configuration — three levels of control, always available

The framework assumes sensible defaults, so you only need to write configuration when you want something different from the default.

Level	What you do	Example
1. Accept the default	Nothing	Tomcat on port 8080, the auto-configured `JsonMapper`
2. Tweak with properties	One line of config	`server.port=9090`, `spring.jackson.default-property-inclusion=non_null`
3. Replace with your own bean	Write Java — the default backs off (3.1)	Define your own `DataSource` or `JsonMapper`

Concretely, in code:

```
// Level 3 — define your own bean → the default backs off
@Bean
JsonMapper jsonMapper() {
    return JsonMapper.builder().findAndAddModules().build();
}

// Level 2 — tweak with properties, the easiest and most common
//    spring.jackson.* , server.* , spring.mvc.* …

// A fourth, blunter option: turn an auto-configuration off entirely
@SpringBootApplication(exclude = SomeAutoConfiguration.class)
// or:  spring.autoconfigure.exclude=com.example.SomeAutoConfiguration
```

---

4. Pillar 3 — Embedded server & the executable jar

Spring Boot normally brings **embedded Tomcat** as part of the web setup.

- The server is **a library your app starts**, not a platform your app is deployed into. It got onto the classpath because it's bundled inside the web starter (pillar 1) — this pillar and pillar 1 are the same mechanism, seen from a different angle.
- Result: *same artifact* runs on a laptop, in CI, in a container. Deployment = "run a jar".

```
./mvnw clean package
java -jar target/shop-0.0.1-SNAPSHOT.jar
java -jar target/shop-0.0.1-SNAPSHOT.jar --server.port=9090
```

Server choices (Boot 4)

Tomcat is the default. To use **Jetty** instead, swap the server starter:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

Still need a WAR? You can (Boot supports it), but that's the exception — see the JSP callout in section 1. Prefer the executable jar.

---

5. Worked examples — no code, just cause and effect

You add…	Boot creates for you
a **web** starter	`DispatcherServlet`, handler mappings, the JSON converter, an **embedded Tomcat**
+ a **JPA** starter + **H2** driver	a `DataSource` (in-memory DB), `EntityManagerFactory`, transaction manager, implementations of your repository interfaces
+ the **actuator** starter	health and metrics endpoints (Session 2)
+ the **security** starter	a security filter chain that protects every endpoint, plus a generated password
`spring.datasource.url=jdbc:postgresql://…`	the same `DataSource`, but pointing to PostgreSQL
your own `DataSource` bean	Boot's `DataSource` **backs off**; yours is used

Notice what did **not** happen: you never wrote XML, never registered a servlet, never wrote a `@Bean` for these. You changed the **inputs** (dependencies, properties) and the **result** changed — exactly the pipeline above.

---

6. The main class and what happens at startup

6.1 The main class

```
package com.example.shop;

@SpringBootApplication
public class ShopApplication {

    public static void main(String[] args) {
        SpringApplication.run(ShopApplication.class, args);
    }
}
```

`@SpringBootApplication` is a **shortcut for three annotations**:

```
@SpringBootConfiguration   // this class is a @Configuration (it can declare
@Bean methods)
@EnableAutoConfiguration   // turn on auto-configuration (pillar 2, section 3)
@ComponentScan             // find @Component/@Service/@Controller… below this
package
```

⚠ **The #1 beginner bug: component scanning `@ComponentScan` starts in the package of the main class and scans downward.**

```
com.example.shop.ShopApplication          ✅ scanned
com.example.shop.product.ProductController ✅ scanned
com.example.other.SomeController          ❌ NOT scanned (sibling
package)
```

Symptom: `404` for a controller that "obviously exists", or `NoSuchBeanDefinitionException`. **Keep the main class in the root package.**

6.2 `SpringApplication.run()` is the actual bootstrap process:

1. Creates an `ApplicationContext` (the Spring IoC container)
2. Reads the `@SpringBootApplication` class to figure out where to component-scan and which auto-configurations to evaluate
3. Runs all the conditional auto-configuration classes, activating the ones whose conditions are satisfied
4. Registers all discovered `@Component`-family beans
5. Starts the embedded servlet container (Tomcat, by default, if it's a web app)
6. Publishes application events (`ApplicationStartedEvent`, etc.) that things like `CommandLineRunner` beans can hook into

Read the startup log — it tells the story:

```
Starting ShopApplication using Java 21 ...
No active profile set, falling back to 1 default profile: "default"
Tomcat initialized with port 8080 (http)
Starting Servlet engine: [Apache Tomcat/11.x]
Root WebApplicationContext: initialization completed in 612 ms
Tomcat started on port 8080 (http) with context path '/'
Started ShopApplication in 1.4 seconds
```

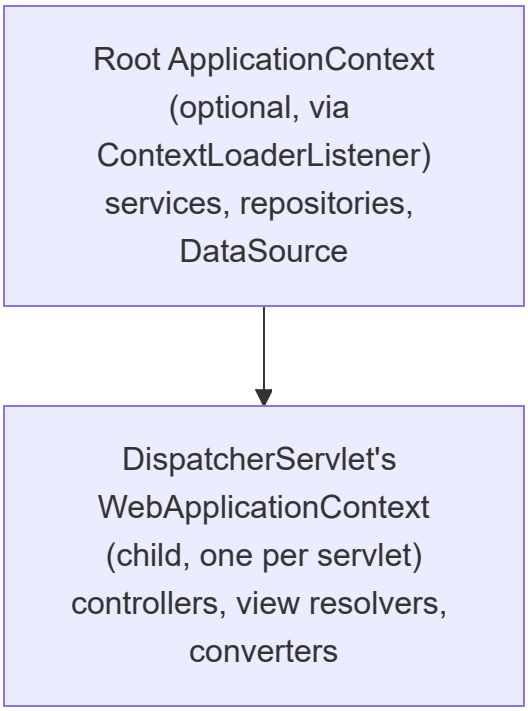
Live demo — "see all the beans nobody wrote"

```
@Bean
CommandLineRunner listBeans(ApplicationContext ctx) {
    return args -> Arrays.stream(ctx.getBeanDefinitionNames())
            .sorted()
            .forEach(System.out::println);   // demo only — use a logger
in real code
}
```

Run it. Students will see `dispatcherServlet`, `jsonMapper`, `requestMappingHandlerMapping`, `tomcatServletWebServerFactory` … **None of them are in our code.** That is pillar 2 at work, using what pillar 1 put on the classpath.

6.3 One context, not two

In plain Spring MVC you typically built a **context hierarchy**:



- The child can see the parent's beans; the parent **cannot** see the child's.

In Spring Boot, by default there is **one** `ApplicationContext` that holds everything:

Question	Plain Spring MVC	Spring Boot
Who creates the context?	The servlet container (listener / servlet init)	`SpringApplication.run`
How many contexts?	0–1 root + 1 per `DispatcherServlet`	**1**
Where do controllers live?	In the servlet's child context	In the same context as everything else
Component scanning	Must be split so root and web don't overlap	Once, from the main class

Why it works: the direction of control is flipped. In MVC the *container* created the contexts. In Boot, `main` creates the context and **the context starts Tomcat**. `DispatcherServletAutoConfiguration` registers `dispatcherServlet` as a normal bean, so it is handed the existing context and uses it instead of creating a child.

---

7. `application.yml` — the basics

is a **configuration file used by Spring Boot** to define application properties.

7.1 Syntax

Same data, two syntaxes:

```
# application.properties
server.port=8081
spring.application.name=shop
```

```
# application.yml
server:
  port: 8081
spring:
  application:
    name: shop
```

A few properties you'll use immediately:

Property	Purpose
`server.port`	HTTP port (`0` = random free port, handy for tests)
`spring.application.name`	Logical name
`spring.jackson.*`	JSON behavior

7.2 Reading a value with `@Value`

For one or two values, `@Value` is enough:

```
@Value("${shop.name}")
private String shopName;

@Value("${shop.currency:USD}")          // default after the colon
private String currency;
```

---

8. Now build it — the project

You now know what each part *does*. Here is how the concepts appear in a real project. Don't memorize the steps.

Concept	Where it appears in the project
Starters + BOM (section 2)	`pom.xml` : parent + starters, no versions
Auto-configuration (section 3)	Nothing — it happens at startup
Embedded server (section 4)	Inside the web starter; built into the jar by the Maven plugin
Pillars together (section 5)	The running app itself
Main class (section 6)	`ShopApplication.java`
`application.yml` (section 7)	`src/main/resources/application.yml`

8.1 Use start.spring.io

Go to **start.spring.io** (or your IDE's "New Spring Project"):

Field	Choose
Project	Maven (or Gradle — the concepts are identical)
Language	Java
Spring Boot	latest **4.x** release (not a SNAPSHOT/milestone)
Packaging	Jar
Java	21 or 25 (LTS)
Dependencies	**Spring Web** (→ `spring-boot-starter-webmvc`)

8.2 The generated `pom.xml`

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>        <!-- use the latest 4.x -->
    <relativePath/>
</parent>

<groupId>com.example</groupId>
<artifactId>shop</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
    <java.version>21</java.version>
</properties>

<dependencies>
```

```
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

This is section 2.3 (starters + BOM) made real: **no `<version>` on any dependency**, because `spring-boot-starter-parent` → `spring-boot-dependencies` already picked compatible ones.

Piece	Job
`spring-boot-starter-parent`	Imports the BOM, sensible plugin config, UTF-8, `-parameters` compiler flag, resource filtering.
`spring-boot-starter-webmvc`	Spring MVC + JSON support (Jackson) + an embedded server (Tomcat by default).
`spring-boot-starter-webmvc-test`	JUnit, Mockito, AssertJ, MockMvc support — the **test companion** of the web starter.
`spring-boot-maven-plugin`	Builds the **executable "fat" jar** and adds `spring-boot:run`.

Lab — "Task Tracker" REST API (Spring Boot web only, in-memory storage)

Goal

Practice the full Session 1 loop — starters, auto-configuration, the embedded server, the main class, `application.yml`, and `@Value` — with **zero persistence**. Storage is a `Map` inside the controller (or a small in-memory "store" class), same spirit as the `ProductController` example from the session.

Domain — one entity, plain Java, no annotations needed

`Task`

Field	Type	Notes
`id`	`Long`	generated in-memory (e.g. `AtomicLong`)
`title`	`String`	required, not blank
`description`	`String`	optional
`completed`	`boolean`	default `false`
`dueDate`	`LocalDate`	optional

Functional requirements — endpoints (unchanged)

Method	Path	Behavior	Status codes
`POST`	`/api/tasks`	Create a task from the JSON body	`201` + `Location` header
`GET`	`/api/tasks`	List all tasks	`200`
`GET`	`/api/tasks?completed=true`	Filter by completion	`200` (empty list if none match)
`GET`	`/api/tasks/{id}`	Fetch one task	`200`, or `404`
`PUT`	`/api/tasks/{id}`	Replace a task's fields	`200`, or `404`
`PATCH`	`/api/tasks/{id}/complete`	Mark completed	`200`, or `404`
`DELETE`	`/api/tasks/{id}`	Delete a task	`204`, or `404`

New requirement — `@Value`-driven configuration

Add these to `application.yml`, and read them with `@Value`, not hardcoded:

yaml

```yaml
tasktracker:
  max-tasks: 50
```

```
  default-page-size: 10
```

1. `POST /api/tasks` must reject creation with `409 Conflict` once the store already holds `maxTasks` tasks — forces you to read `maxTasks` at request time, not just inject it and ignore it.
2. `GET /api/tasks` must support `?limit=` as an optional query param, falling back to `defaultPageSize` when the caller doesn't pass one.